Метапрограммирование

Коновалов И.В.

Цикл жизни приложения

- **разработка**

  Работа с исходным кодом

- **тестирование**

  Запуск приложения с возможностью сопоставить с исходным кодом

- **эксплуатация**

  Работа приложения в реальных условиях

Цикл жизни приложения

- **разработка** **compile time**

  Работа с исходным кодом

- **тестирование** **runtime**

  Запуск приложения с возможностью сопоставить с исходным кодом

- **эксплуатация**

  Работа приложения в реальных условиях

Что такое метапрограммирование

- программирование – эффект, который случится в runtime

```
printf("Hello, world\");
```

- метапрограммирование – эффект, который случится в compile time

```
#ifdef _DEBUG
    #define DEBUG(msg...) fprintf(stderr, msg...)
#else
    #define DEBUG(msg)
#endif
```

Кодогенерация средствами языка

```
(defmacro ore-te-with-ctx (path &rest body)
  (let ((ctx (make-symbol "ctx")))
    `(let* ((,ctx (ore-te--get-ctx ,path))
            (org-id-locations (plist-get ,ctx  :locations))
            (org-id-files (plist-get ,ctx  :files))
            (org-id-locations-file (plist-get ,ctx  :loc-file)))
       ,@body
       (plist-put ,ctx  :locations org-id-locations)
       (plist-put ,ctx  :files org-id-files))))
```

Препроцессирование

```
#define BUF_SIZE 1024

char buf[BUF_SIZE];
```

препроцессор

```
char buf[1024];
```

Препроцессирование

```
#define HOOK_CALL(lib, name, size)                                          \
static void (*_ ## name ## _hook)(void);                                    \
void name (void) {                                                          \
        void *args = __builtin_apply_args();                                \
        DBG("Calling %s", #name);                                           \
        if (!_ ## name ## _hook) {                                          \
                void *orig_p, *orig;                                        \
                void *hook = get_lib_hook(lib);                             \
                orig = dlsym(RTLD_NEXT, #name);                             \
                if (!orig) {                                                \
                        void *orig_lib = dlopen(lib_hook_list[lib].filename, RTLD_LAZY | RTLD_GLOBAL);\
                        if (!orig_lib) {                                    \
                                fprintf(stderr, "Can't open library %s: %s\n", lib_hook_list[lib].filename, dlerror());\
                                exit(-1);                                   \
                        }                                                   \
                        orig = dlsym(orig_lib, #name);                      \
                        if (!orig) {                                        \
                                fprintf(stderr, "Symbol not found "#name":%s\n", dlerror());\
                                exit(-1);                                   \
                        }                                                   \
                }                                                           \
                if (hook) {                                                 \
                        orig_p = dlsym(hook, "_" #name);                    \
                        if (orig_p) {                                       \
                                *(void **) orig_p = orig;                   \
                                _ ## name ## _hook = dlsym(hook, #name);\
                        }                                                   \
                }                                                           \
                if (!_ ## name ## _hook) {                                  \
                        DBG("Can't hook %s, using original", #name);        \
                        _ ## name ## _hook = orig;                          \
                }                                                           \
        }                                                                   \
        __builtin_return(__builtin_apply(_ ## name ## _hook, args, size));\
}
```

https://gist.github.com/aragaer/1072069

Инструменты кодогенерации

- version.c.in

```
const char version[] = "@VERSION@";
```

- autoconf/automake
- cmake

```
configure_file(version.c.in version.c)
```

- version.c

```
const char version[] = "1.2.3";
```

Ручная кодогенерация

```
version.c:
    git describe --tags | \
        sed 's/^/const char version[] = \"/;s/-.*/\";/' > $@
```

Вычисления времени компиляции

```
char message[] = "Hello, world";

size_t message_len = 12; // OK, но нехорошо

size_t message_len = strlen(message); // ОШИБКА

size_t message_len = sizeof(message) - 1; // OK
```

Вычисления времени компиляции

```
char message[1024] = "Hello, ";
size_t message_prefix_len = 7; //  strlen(message)



strncat(message + message_prefix_len, username,
        sizeof(message) - message_prefix_len);
```

Вычисления времени компиляции

```
template<size_t N>
constexpr size_t cstrlen(const char (&)[N]) noexcept {
    return N - 1;
}
char message[1024] = "Hello, ";
size_t message_len = cstrlen("Hello, "); // OK
```

Вычисления времени компиляции

```
template<size_t N>
constexpr size_t cstrlen(const char (&)[N]) noexcept {
    return N - 1;
}
char message[1024] = "Hello, ";
size_t message_len = cstrlen("Hello, "); // OK
size_t cstrlen<20>(char (&)[20]); // инстанцирование
```

Вычисления времени компиляции

```
template<size_t N>
constexpr size_t cstrlen(const char (&)[N]) noexcept {
    return N - 1;
}
char message[1024] = "Hello, ";
size_t message_len = cstrlen<8>("Hello, "); // OK
size_t cstrlen<20>(char (&)[20]); // инстанцирование
```

Вычисления времени компиляции

```
template<size_t N>
constexpr size_t cstrlen(const char (&)[ N+1]) noexcept {
    return N;
}
char message[1024] = "Hello, ";
size_t message_len = cstrlen("Hello, "); // couldn't deduce template
parameter
```

Специализация времени компиляции

```
template<bool poison>
class MyInt {
	int _value;
public:
	void setValue(int value) {
		_value = poison ? value ^ ~0 : value;
	}

	int getValue() {
		return poison ? _value ^ ~0 : value;
	}
};

template class MyInt<true>;
template class MyInt<false>;
```

Условие времени компиляции

```
template<int N>
const char *name() {
        if constexpr (N <= 0)
                return "negative";
        if constexpr (N >= 3)
                return "some big number";
        if constexpr (N == 1)
                return "one";
        return "two";
}
```

```
template const char *name<1>();

template const char *name<10>();

template const char *name<20>();
```

Условие времени компиляции

```
template<int N>
const char *name() {
	if constexpr (N <= 0)
		return "negative";
	if constexpr (N >= 3)
		return "some big number";
	if constexpr (N == 1)
		return "one";
	return "two";
}
```

```
template const char *name<1>();
const char *name<1>() {
	return "one";
}

template const char *name<10>();
const char *name<10>() {
	return "some big number";
}

template const char *name<20>();
const char *name<20>() {
	return "some big number";
}
```

Условие времени компиляции

```
template<int N>
const char *name() {
        if constexpr (N <= 0)
                return "negative";
        if constexpr (N >= 3)
                return "some big number";
        if constexpr (N == 1)
                return "one";
        return "two";
}

template<> const char *name<7>() {
        return "a lucky number";
}
```

```
template const char *name<1>();
const char *name<1>() {
        return "one";
}

template const char *name<10>();
const char *name<10>() {
        return "some big number";
}

template const char *name<20>();
const char *name<20>() {
        return "some big number";
}
```

Рекурсивные шаблоны

```
template<int N>
struct Fib {
	static constexpr int value = Fib<N-1>::value + Fib<N-2>::value;
};

template<> struct Fib<0> {
	static constexpr int value = 0;
};

template<> struct Fib<1> {
	static constexpr int value = 1;
};

printf("%d\n", Fib<10>::value); // значение 55 вычислено еще на этапе компиляции
```

Параметрический полиморфизм

```
template<typename T>
class DiscreteSetSampler {
	std::vector<T> _values;
	std::vector<float> _weights;
public:
	...
};
```

Несколько параметров

```
template<typename T, int N, int M>
T **make_matrix() {
        return new T[N][M];
}
```

```
template<> int **make_matrix<int, 10, 20>();
```

```
template<typename T1, typename T2>
class MyPair {
private:
        T1 _first;
        T2 _second;
...
};
```

```
template class MyPair<int, char *>;
```

Переменное число параметров

```
template<typename Result, typename Args...>
class Caller<Result, std::tuple<Args...>> {
public:
	virtual Result Call(Args... args) = 0;
};
```

```
class Adder : public Caller<int, std::tuple<int, int>> {
public:
	virtual int Call(int x, int y) {
		return x + y;
	}
};
```

Свойства типов

```
template<typename T, typename = void>
struct has_tag : std::false_type {
	static inline int tag(T *const obj) {
		return 0;
	}
};

template<typename T>
struct has_tag<T, decltype((void T::tag), void())> : std::true_type {
	static inline int tag(T *const obj) {
		return obj->tag;
	}
};
```

```
struct S1 {
	int x;
} s1 = { 10 } ;

struct S2 {
	int tag;
} s2 = { 20 };


has_tag<S1>::tag(&s1); // 0
has_tag<S2>::tag(&s2); // 20
```

Свойства типов

```
template<typename T, typename = void>
struct has_tag : std::false_type {
	static inline int tag(T *const obj) {
		return 0;
	}
};

template<typename T>
struct has_tag<T, decltype((void T::tag), void())> : std::true_type {
	static inline int tag(T *const obj) {
		return obj->tag;
	}
};
```

```
struct S1 {
	int x;
} s1 = { 10 } ;

struct S2 {
	int tag;
} s2 = { 20 };
```

```
has_tag<S1>::tag(&s1); // 0
has_tag<S2>::tag(&s2); // 20
```

Substitution Failure Is Not An Error

Десятое правило Гринспена

https://ru.wikipedia.org/wiki/Десятое_правило_Гринспена

Любая достаточно сложная программа на Си или Фортране содержит заново написанную, неспецифированную, глючную и медленную реализацию половины языка Common Lisp.